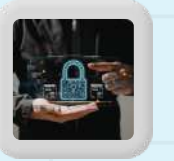


SYSTEMS SECURITY

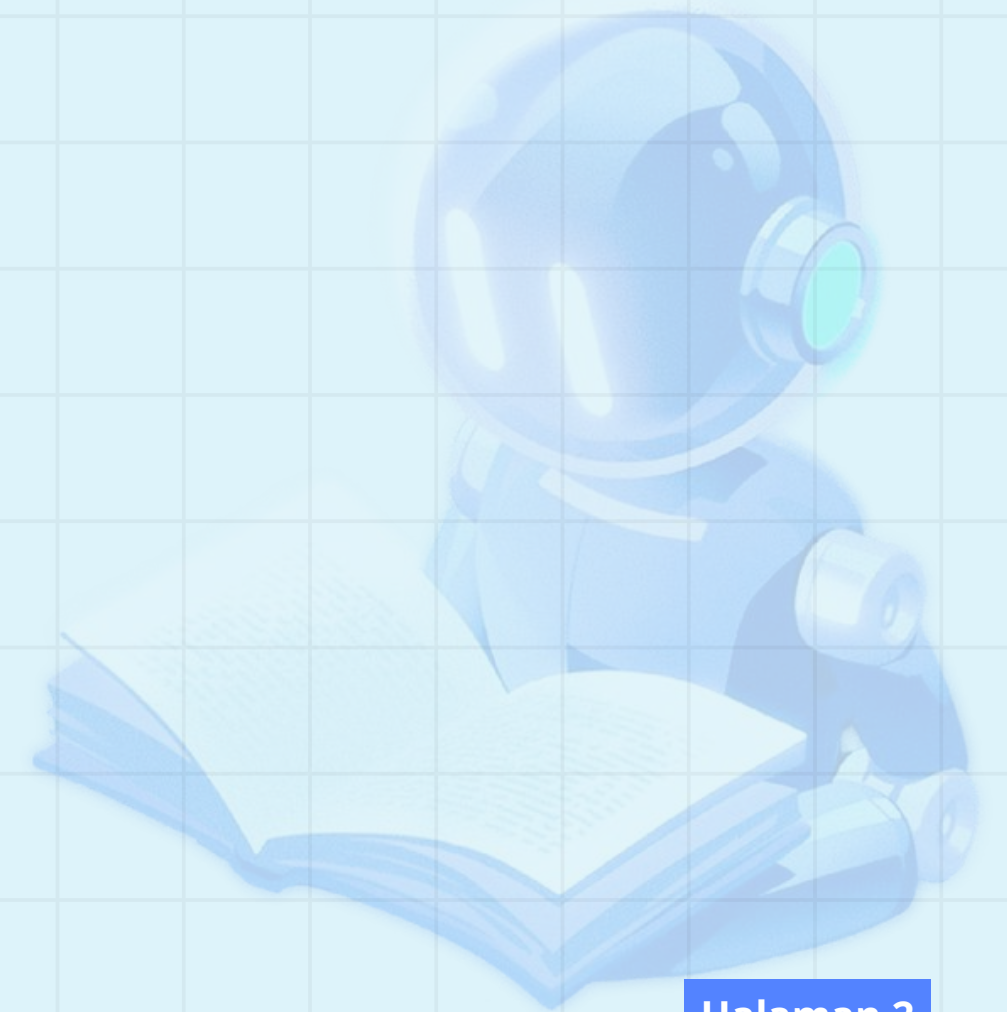
ANALISIS KOMPREHENSIF BERDASARKAN THE CYBER SECURITY BODY OF KNOWLEDGE (CYBOK)

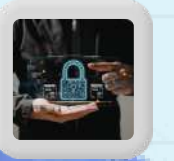
Mata Kuliah: **JSI62126 – Keamanan Informasi**
Dosen Pengampu: **Fajril Akbar, M.Sc**



NAMA ANGGOTA KELOMPOK 6:

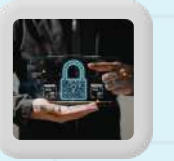
1. Dwigo Fajar Briliano (2411521014)
2. Zahra Aulia Nasution (2411521015)
3. Ahmad Faiz Batubara (2411521016)
4. Diva Ramadhani (2411521017)
5. Fuadi Dhiyaulhaq (2411522001)
6. Aqila Higeneea Taufik (2411522003)





CRYPTOGRAPHY

Referensi: CyBOK Knowledge Area Version 1.0.2 (Dieter Gollmann)

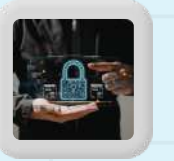


APA ITU CRYPTOGRAPHY ?

Pengaman informasi agar tidak mudah disadap, diubah, atau disalahgunakan oleh pihak lain.

Cryptography (Kriptografi):

Teknik atau ilmu yang digunakan untuk melindungi informasi dengan cara mengubah data asli menjadi bentuk yang tidak dapat dibaca oleh pihak yang tidak berwenang. Tujuannya menjaga keamanan data saat disimpan maupun saat dikirim melalui jaringan.



SECURITY GOALS AND ATTACK MODELS

dalam kriptografi mencakup tiga aspek utama yaitu confidentiality, integrity, dan availability.

Confidentiality memastikan bahwa informasi hanya dapat diakses oleh pihak yg berwenang.

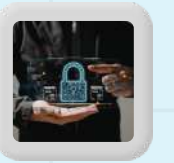
Integrity menjamin bahwa data tidak diubah atau dimodifikasi tanpa izin.

Availability memastikan bahwa sistem dan data tetap dapat diakses ketika dibutuhkan.

Dalam analisis keamanan kriptografi juga digunakan **model serangan** seperti Chosen Plaintext Attack (CPA) dan Chosen Ciphertext Attack (CCA) untuk menguji ketahanan algoritma terhadap upaya penyerangan.

CPA adalah model serangan dalam kriptografi di mana penyerang dapat memilih plaintext (pesan asli) tertentu untuk dienkripsi, lalu melihat hasil ciphertext-nya.

CCA adalah model serangan di mana penyerang dapat memilih ciphertext tertentu dan meminta sistem untuk mendekripsinya menjadi plaintext.



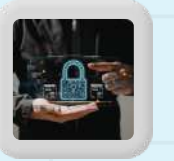
SYMMETRIC CRYPTOGRAPHY

Adalah metode enkripsi yang menggunakan satu kunci yang sama untuk proses enkripsi dan dekripsi, lebih cepat dan efisien untuk mengamankan data dalam jumlah besar. Beberapa jenis utama dalam kriptografi simetris adalah stream cipher dan block cipher.

Stream cipher adalah jenis algoritma dalam kriptografi simetris yang mengenkripsi data satu per satu (bit atau byte) secara berurutan.

Block cipher adalah metode enkripsi yang bekerja dengan cara membagi data menjadi blok-blok dengan ukuran tertentu, kemudian setiap blok dienkripsi menggunakan kunci yang sama.

Advanced Encryption Standard (AES) adalah salah satu algoritma block cipher yang paling banyak digunakan di dunia. AES digunakan untuk mengamankan berbagai sistem seperti jaringan internet, transaksi perbankan, dan penyimpanan data. Algoritma ini dikenal sangat kuat dan efisien serta mendukung ukuran kunci seperti 128 bit, 192 bit, dan 256 bit.



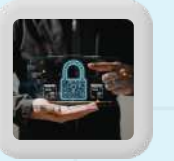
HASH AND MESSAGE AUTHENTICATION CODES (MAC)

digunakan untuk menjaga integritas data, karena perubahan kecil pada data akan menghasilkan nilai hash yang berbeda.

HASH adalah algoritma yang mengubah data dengan panjang apa pun menjadi nilai tetap yang disebut hash value.

Message Authentication Codes (MAC) digunakan untuk memastikan bahwa pesan benar-benar berasal dari pengirim yang sah serta tidak mengalami perubahan selama proses pengiriman.

Kegunaan utama hash adalah untuk menjaga integritas data, yaitu memastikan bahwa data tidak berubah selama penyimpanan atau pengiriman. Jika data diubah sedikit saja, nilai hash yang dihasilkan juga akan berubah, sehingga perubahan tersebut dapat dengan mudah dideteksi. Fungsi hash sering digunakan dalam penyimpanan password, verifikasi file, dan sistem keamanan data.



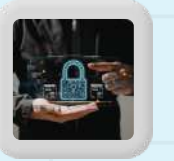
PUBLIC KEY CRYPTOGRAPHY

Kriptografi kunci publik menggunakan dua kunci yang berbeda, kunci publik untuk enkripsi dan kunci privat untuk dekripsi. Pendekatan ini memudahkan proses komunikasi aman tanpa harus berbagi kunci rahasia terlebih dahulu. algoritma terkenal antara lain metode Diffie–Hellman , RSA, serta (ECC).

Diffie–Hellman key exchange adalah metode kriptografi yang digunakan untuk menukar kunci rahasia secara aman melalui jaringan yang tidak aman. Metode ini memungkinkan dua pihak membuat kunci bersama tanpa harus mengirimkan kunci tersebut secara langsung.

RSA adalah algoritma kriptografi kunci publik yang digunakan untuk enkripsi data dan pembuatan tanda tangan digital. RSA bekerja menggunakan dua kunci, yaitu kunci publik untuk mengenkripsi pesan dan kunci privat untuk mendekripsinya.

Elliptic Curve Cryptography (ECC) adalah metode kriptografi kunci publik yang menggunakan konsep matematika kurva eliptik. ECC dapat memberikan tingkat keamanan yang tinggi dengan ukuran kunci yang lebih kecil dibandingkan algoritma lain seperti RSA.

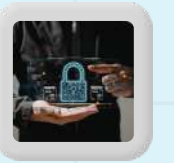


DIGITAL SIGNATURES

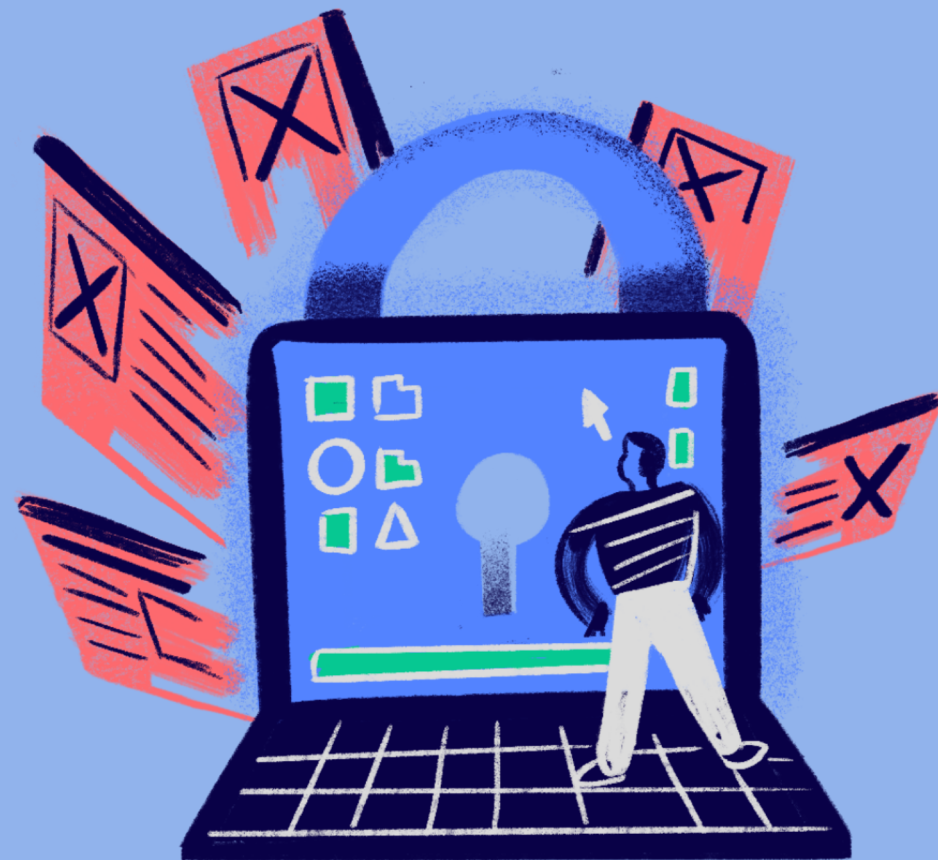
Tanda tangan digital digunakan untuk memverifikasi keaslian dan integritas suatu pesan atau dokumen digital. Dengan menggunakan algoritma kriptografi kunci publik. Dengan tanda tangan digital, penerima dapat memastikan bahwa pesan benar-benar berasal dari pengirim yang sah dan tidak mengalami perubahan selama proses pengiriman. Teknologi ini banyak digunakan dalam transaksi elektronik, pengiriman email aman, serta validasi dokumen digital.



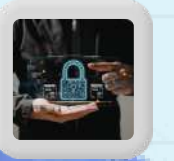
Beberapa algoritma yang digunakan dalam tanda tangan digital antara lain RSA dan Elliptic Curve Cryptography (ECC).



FUTURE DIRECTIONS: POST-QUANTUM CRYPTOGRAPHY

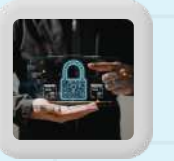


Perkembangan komputer kuantum dapat menjadi ancaman bagi sistem kriptografi yang digunakan saat ini. Algoritma seperti RSA dan Elliptic Curve Cryptography berpotensi lebih mudah dipecahkan oleh komputer kuantum karena kemampuannya melakukan perhitungan yang sangat kompleks dengan cepat. Untuk mengatasi hal tersebut, para peneliti mengembangkan **Post-Quantum Cryptography**, yaitu algoritma kriptografi baru yang dirancang agar tetap aman terhadap serangan komputer kuantum. Tujuannya adalah menjaga keamanan komunikasi digital, transaksi online, dan penyimpanan data di masa depan meskipun teknologi komputasi terus berkembang.



OPERATION SYSTEM SECURITY

Referensi: CyBOK Knowledge Area Version 1.0.2 (Dieter Gollmann)



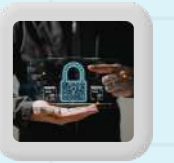
APA ITU OPERATION SYSTEM SECURITY ?

Kerangka keamanan pada level sistem operasi untuk mengisolasi domain keamanan dan memediasi akses terhadap sumber daya sistem.



Operation System Securitiy:

Upaya untuk memastikan keamanan pada level sistem operasi dan hypervisor, serta menjamin keamanan yang diberikan kepada seluruh komponen di atasnya. OS dan hypervisor adalah komponen paling privileged dalam sistem dan berperan krusial dalam membuat sistem menjadi aman atau tidak aman



OS SECURITY GOALS

Sistem operasi diposisikan sebagai fondasi keamanan seluruh komputer. Artinya, semua aplikasi, database, browser, bahkan software keamanan sekalipun berdiri di atas OS.

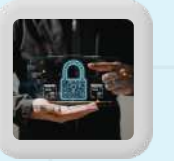
Tujuan Keamanan OS:

1. Confidentiality (Kerahasiaan) → proses tidak boleh membaca memori proses lain
2. Integrity (Integritas) → user tidak boleh memodifikasi kernel
3. Availability (Ketersediaan) → satu proses tidak boleh memonopoli CPU

Security Domain Separation: Kernel vs User Space

Pemisahan kernel vs user space adalah mekanisme paling fundamental untuk mencapai security goals. Kalau ini tidak ada, semua tujuan keamanan runtuh.

- Kernel → privileged mode, akses penuh
- User space → unprivileged mode
- Interaksi melalui system call
- Tujuan: isolasi & kontrol



SECURITY PRIMITIVES

Kata “primitive” di sini bukan berarti primitif dalam arti sederhana, tetapi berarti: Mekanisme dasar yang menjadi fondasi keamanan. Security primitives adalah mekanisme teknis paling dasar yang digunakan OS untuk mewujudkan security goals

Beberapa primitive utama OS meliputi

Authentication & Identification:

Digunakan untuk mengetahui siapa saja yang mengakses sistem

Access Control:

Mengatur siapa yang boleh melakukan apa terhadap suatu resource.

Memory Protection:

Memisahkan memori antar proses dengan tujuan proses tidak bisa baca proses lain

Privilege Separation:

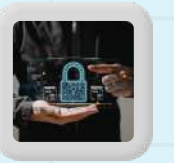
Memisahkan kode biasa dengan kode yang punya hak istimewa

Address Spaces:

cara teknis untuk mewujudkan memory protection. Setiap proses memiliki virtual address space sendiri.

Mediation via System Calls:

User process tidak boleh langsung mengakses hardware atau resource sensitif dan harus melalui system call.



ARCHITECTURE & DESIGN

Arsitektur sistem operasi menentukan bagaimana komponen OS disusun dan bagaimana isolasi antar security domain diterapkan. Desain ini berpengaruh langsung terhadap tingkat keamanan, terutama dalam hal isolasi, besar Trusted Computing Base (TCB), dan attack surface. Perbedaan struktur kernel menghasilkan perbedaan tingkat keamanan dan efisiensi. Secara umum, terdapat tiga pendekatan utama dalam desain OS, yaitu:

Monolithic Kernel

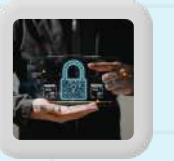
Sebagian besar komponen OS seperti file system, network, dan driver berjalan di kernel space dengan hak akses penuh. Performa tinggi karena komunikasi langsung di dalam kernel. Namun, karena banyak kode berada di mode privileged, bug pada satu komponen dapat berdampak ke seluruh sistem dan memperbesar attack surface
contoh : Linux

Microkernel

Kernel hanya menangani fungsi inti (IPC, scheduling, memori dasar), sementara driver dan service lain berjalan di user space. Desain ini membuat isolasi lebih kuat dan TCB lebih kecil, sehingga risiko keamanan berkurang, meskipun ada sedikit overhead komunikasi
contoh : L4 microkernel

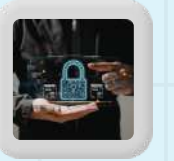
Unikernel

Aplikasi dan komponen OS yang diperlukan digabung menjadi satu image khusus yang berjalan di atas hypervisor. Hanya memuat fungsi yang dibutuhkan sehingga attack surface sangat kecil, tetapi umumnya hanya untuk satu aplikasi dan bukan sistem multi-user.



PERBANDINGAN MONOLITHIC, MICROKERNEL DAN UNIKERNEL

Aspek	Monolithic	Microkernel	Unikernel
Ukuran Kernel	Besar	Kecil	Sangat Minimal
Letak Driver	Kernel Space	User Space	Tergantung Build
Attack Surface	Besar	Lebih kecil	Sangat Kecil
Isolasi	Lebih Terbatas	Lebih Kuat	Tidak Multi-User
Cocok untuk	General Purpose OS	High Assurance System	Cloud & VM



P RINCIPLES OF SECURE DESIGN

Least Privilege

Setiap proses atau user hanya boleh memiliki hak akses minimum yang diperlukan untuk menjalankan tugasnya.
Contoh dalam OS: Aplikasi berjalan di user mode, bukan kernel mode.

Economy of Mechanism

Desain sistem keamanan harus sesederhana mungkin karna Sistem yang kompleks lebih sulit diverifikasi dan lebih mudah mengandung bug.
Contoh dalam OS:

- Microkernel memiliki kernel yang kecil.

Complete Mediation

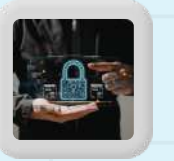
Setiap akses ke resource harus diperiksa.
Dalam OS:

- Semua akses file melalui system call.
- Kernel selalu mengecek permission sebelum memberi akses.
- Tidak boleh ada bypass.

Fail-Safe Defaults

Secara default, akses harus ditolak kecuali secara eksplisit diizinkan.
Contoh:

- File tanpa permission = tidak bisa diakses.
- Firewall default deny.
- User baru tidak otomatis admin.



MEMORY PROTECTION & SANDBOXING

Memory protection adalah mekanisme yang memastikan:

- Proses tidak bisa mengakses memori proses lain
- User space tidak bisa mengakses kernel space
- Akses memori bisa dibatasi (read/write/execute)

Pada sistem seperti Linux dan Microsoft Windows, ini dilakukan melalui Virtual Memory, Memory Management Unit, dan Page Table Permission

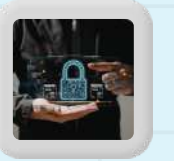
Sandboxing membatasi kemampuan aplikasi secara keseluruhan. Sandbox biasanya membatasi:

- Akses file
- Akses jaringan
- Akses sistem call tertentu
- Akses device

Contoh:

- Browser berjalan dalam sandbox

Dengan kombinasi memory protection dan sandboxing, sistem operasi menerapkan prinsip least privilege dan isolation secara nyata. Mekanisme ini menjadikan kegagalan atau kompromi pada satu aplikasi tidak secara otomatis mengorbankan seluruh sistem

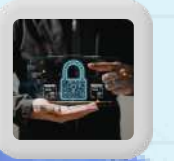


FUNDAMENTAL SECURITY IN LINUX AND WINDOWS

Sistem operasi seperti Linux dan Microsoft Windows menerapkan prinsip keamanan yang serupa dengan pendekatan berbeda. Linux menggunakan model UID, GID, serta permission read, write, dan execute untuk mengatur akses terhadap file dan proses. Pemisahan tegas antara user biasa dan root mencerminkan prinsip least privilege, sementara isolasi proses melalui virtual memory serta pemisahan kernel dan user space menjaga agar satu proses tidak dapat mengakses atau merusak proses lain secara langsung.



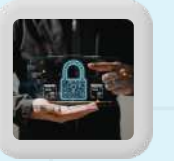
Windows menerapkan Access Control List (ACL) yang lebih granular untuk mengatur hak akses setiap objek sistem. Mekanisme seperti User Account Control (UAC) membatasi penggunaan hak administrator guna mencegah penyalahgunaan privilege. Selain itu, pemisahan user mode dan kernel mode serta perlindungan tambahan pada kernel menunjukkan bahwa kedua sistem ini dibangun di atas fondasi yang sama, yaitu isolasi, kontrol akses ketat, dan pembatasan privilege sebagai inti keamanan sistem operasi modern.



VS

VIRTUALISATION & HARDWARE SECURITY

Referensi: CyBOK Knowledge Area Version 1.0.2 (Dieter Gollmann)



VIRTUALISATION SECURITY

Virtualisation security membahas bagaimana keamanan dijaga ketika beberapa sistem berjalan pada satu perangkat keras yang sama. Virtualisasi memungkinkan efisiensi resource, namun juga menciptakan tantangan baru karena adanya resource sharing.

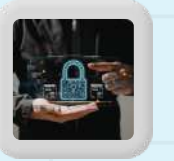
Komponen Virtualisasi

- Virtual Machine (VM) → sistem operasi virtual independen
- Container → lingkungan ringan berbagi kernel
- Hypervisor → pengelola resource hardware

Tujuan Keamanan

- Menjaga isolasi antar lingkungan virtual
- Mencegah akses data lintas VM
- Mengontrol penggunaan resource bersama

Walaupun terlihat terpisah, seluruh sistem tetap menggunakan hardware yang sama sehingga isolasi menjadi aspek keamanan paling penting.



HYPERVISOR & ISOLATION MODEL

Hypervisor adalah software yang mengelola virtual machine dan membagi resource hardware. Jenisnya dibedakan berdasarkan cara kerjanya terhadap sistem operasi dan hardware.

Model isolasi memastikan setiap VM tetap terpisah walaupun menggunakan hardware yang sama, sehingga tidak dapat saling mengakses atau mengganggu.

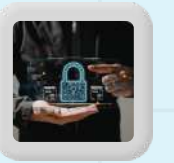
1. Type 1 (Bare-metal)

berjalan langsung di hardware sehingga performanya lebih tinggi dan lebih aman karena tidak bergantung pada sistem operasi.

2. Type 2 (Hosted)

berjalan di atas sistem operasi utama seperti aplikasi biasa, lebih mudah digunakan tetapi keamanannya bergantung pada OS host.

- Pemisahan memori antar VM memastikan data tiap sistem tidak dapat dibaca oleh VM lain.
- Kontrol akses perangkat keras mengatur penggunaan CPU, storage, dan jaringan secara aman.
- Manajemen hak akses sistem membatasi izin setiap VM agar tidak melebihi kewenangannya.



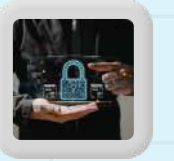
HARDWARE VULNERABILITIES

Kerentanan keamanan tidak hanya berasal dari software, tetapi juga dapat muncul dari desain hardware. Optimasi performa pada hardware modern terkadang membuka peluang terjadinya kebocoran informasi.

Bentuk Kerentanan Hardware

- Cache sharing antar proses
- Perbedaan waktu eksekusi instruksi
- Penggunaan resource bersama

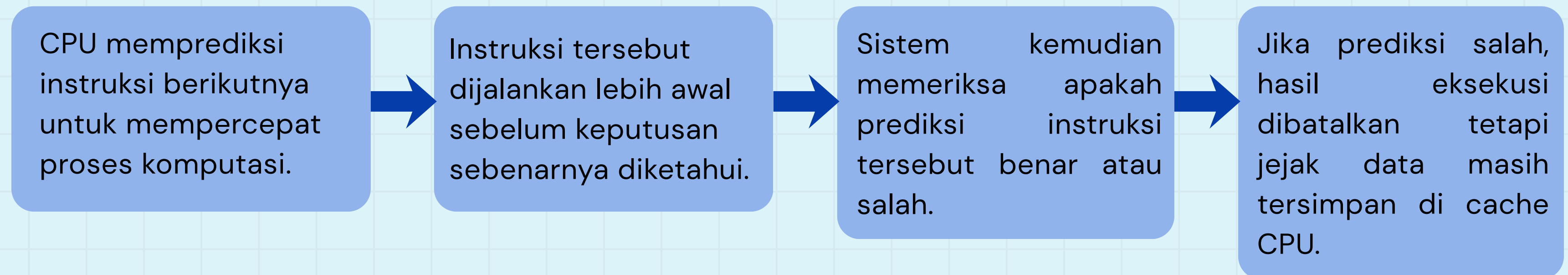
Side-channel attack memperoleh informasi dari efek samping sistem, seperti waktu respon atau aktivitas cache, tanpa mengakses data secara langsung. Hal ini menunjukkan bahwa isolasi software saja tidak cukup jika hardware masih membocorkan informasi.



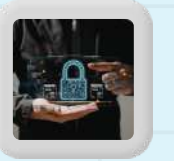
SPECULATIVE EXECUTION ISSUES

Speculative execution adalah teknik optimasi CPU yang memprediksi instruksi berikutnya agar proses komputasi lebih cepat. Namun teknik ini meninggalkan jejak data sementara pada cache prosesor.

Cara Kerja Speculative Execution



Serangan seperti Spectre dan Meltdown memanfaatkan jejak tersebut untuk membaca data dari area memori yang seharusnya terlindungi.



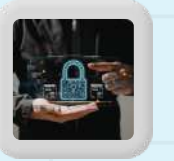
TRUSTED EXECUTION ENVIRONMENTS (TEE)

Trusted Execution Environment (TEE) adalah lingkungan aman di dalam prosesor yang digunakan untuk menjalankan proses sensitif secara terisolasi dari sistem operasi utama. TEE memindahkan perlindungan keamanan langsung ke hardware sehingga meningkatkan tingkat kepercayaan sistem.

Fungsi Trusted Execution Environment

- Menjalankan operasi sensitif dalam lingkungan yang terisolasi.
- Melindungi data penting dari akses sistem operasi utama.
- Mengamankan proses kriptografi dan autentikasi pengguna.

Contoh implementasi Trusted Execution Environment meliputi Intel SGX yang menyediakan enclave aman untuk menjalankan aplikasi sensitif, serta ARM TrustZone yang memisahkan area sistem normal dan area aman pada perangkat mobile untuk melindungi data penting.



HARDWARE ROOT OF TRUST

Hardware Root of Trust adalah konsep keamanan yang memastikan sistem dapat dipercaya sejak perangkat pertama kali dinyalakan. Keamanan dibangun secara bertahap mulai dari hardware hingga sistem operasi melalui proses verifikasi berlapis.

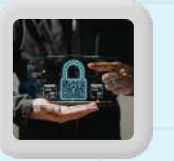
komponen utama

Secure Boot memastikan hanya software yang memiliki tanda tangan terpercaya yang dapat dijalankan saat proses booting, sehingga mencegah malware aktif sejak awal sistem menyala.

Trusted Platform Module (TPM) adalah chip keamanan khusus yang menyimpan kunci kriptografi secara aman dan digunakan untuk memverifikasi integritas sistem.

Tujuan Hardware Root of Trust

- Menjaga integritas sistem sejak proses booting dimulai.
- Mencegah program berbahaya berjalan saat startup.
- Membentuk chain of trust dari hardware hingga sistem operasi.



MODERN VIRTUALISATION

Modern virtualisation merupakan perkembangan teknologi virtualisasi yang berfokus pada efisiensi, skalabilitas, dan isolasi ringan untuk mendukung lingkungan cloud dan aplikasi modern.

komponen utama

Container-Based Virtualisation

Container menjalankan aplikasi dengan berbagi kernel sistem operasi sehingga lebih ringan dibanding virtual machine.

Cloud & Microservices Environment

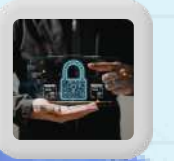
Virtualisasi modern mendukung arsitektur microservices yang membutuhkan deployment cepat dan fleksibel.

Security Challenges

Walaupun lebih efisien, isolasi yang lebih ringan meningkatkan risiko misconfiguration dan serangan antar container.

Lightweight Isolation

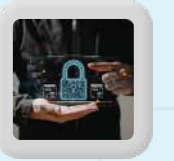
Teknologi baru menggunakan isolasi minimal namun tetap menjaga keamanan antar aplikasi.



DS

DISTRIBUTED SYSTEM SECURITY

Referensi: CyBOK Knowledge Area Version 1.0.2 (Dieter Gollmann)



KARAKTERISTIK & TANTANGAN UTAMA SISTEM TERDISTRIBUSI

Komposisi sumber daya (komputasi & komunikasi) yang tersebar secara geografis untuk menyediakan layanan yang andal dan tersedia tinggi.

Tantangan Sinkronisasi

Memerlukan koordinasi antar komponen yang tersebar melalui middleware untuk gaya komunikasi seperti message passing atau RPC.

Replikasi Data

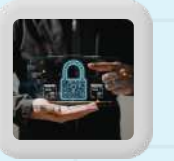
Penggunaan skema replikasi untuk menangani kegagalan sumber daya dan memastikan akses data yang konsisten.

Konsensus

Proses mencapai kesepakatan nilai antar proses yang tersebar, dengan properti utama: Agreement (semua setuju nilai yang sama), Validity, dan Termination.

Model Konsistensi

Spektrum konsistensi dari yang kuat (strong), sekuensial, kausal, hingga konsistensi akhir (**eventual consistency**).



KEAMANAN PADA SISTEM P2P

Skalabilitas tinggi, ketahanan terhadap kegagalan node individu, dan tidak adanya koordinasi terpusat.

Unstructured P2P

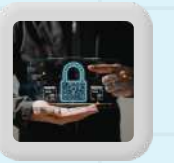
- Tidak memiliki topologi tetap; menggunakan algoritma pencarian seperti flooding atau random walks.
- Cocok untuk penyebaran data (data dissemination) skala besar.

Structured P2P

- Menggunakan Distributed Hash Table (DHT) untuk pemetaan data ke node.
- Efisien dalam penemuan sumber daya (resource discovery) dengan jalur routing yang terukur.

Hybrid P2P

Menggabungkan elemen terstruktur dan tidak terstruktur
Contoh: BitTorrent



SERANGAN PADA SISTEM TERDISTRIBUSI

Skalabilitas tinggi, ketahanan terhadap kegagalan node individu, dan tidak adanya koordinasi terpusat.

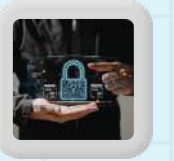
Sybil Attack:

- Penyerang membuat banyak identitas palsu (node Sybil) untuk mendapatkan pengaruh yang tidak proporsional dalam jaringan
- Dapat digunakan untuk menguasai mekanisme voting atau merusak tabel routing.

Eclipse Attack: Varian serangan di mana penyerang mengontrol semua koneksi masuk dan keluar dari node target. Tujuan :

Mengisolasi node dari jaringan utama sehingga penyerang dapat memanipulasi pandangan node tersebut terhadap status sistem.

Dampak CIA: Serangan ini mengancam Ketersediaan (Availability) dengan memutus rute, dan Integritas dengan memberikan data palsu.



KEAMANAN INFRASTRUKTUR SKALA BESAR

Infrastruktur skala besar sering menggunakan coordinated resource clustering untuk memberikan ilusi sistem terpusat.

Cloud Computing

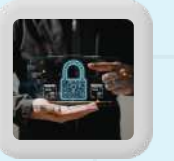
- Menggunakan virtualisasi untuk isolasi sumber daya.

Tantangan:

Keamanan pada alur data, kontrol akses (multitenansi), dan transportasi data antar pusat data.

Ancaman pada Infrastruktur:

- Kompromi pada Scheduler/Broker yang mengelola tugas dan sumber daya.
- Serangan Man-in-the-Middle (MITM) pada jalur komunikasi data yang sedang transit.



RESILIENSI DAN TOLERANSI KESALAHAN

Byzantine Faults

Kegagalan di mana node yang rusak dapat berperilaku secara acak atau jahat (misalnya mengirimkan data yang berbeda ke node yang berbeda).

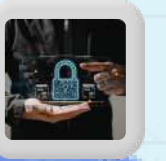
Byzantine Fault Tolerance (BFT):

- Protokol yang menjamin eksekusi operasi yang benar selama maksimal sepertiga ($1/3$) dari total node dikompromi.
- Digunakan dalam sistem yang memerlukan integritas tinggi seperti distributed ledgers (Blockchain).

Fault Tolerance Lainnya:

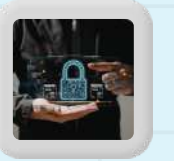
Crash Faults: Menangani node yang berhenti beroperasi secara tiba-tiba.

Two-Phase Commit (2PC) & Three-Phase Commit (3PC): Memastikan atomisitas transaksi meskipun ada kegagalan koordinasi.



A A A AUTHENTICATION, AUTHORISATION, & ACCOUNTABILITY)

Referensi: Authentication, Authorisation & Accountability Knowledge,
CyBOK Knowledge Area Version 1.0.2 (Dieter Gollmann)



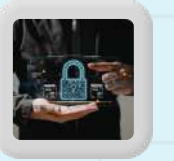
APA ITU AAA?

kerangka kerja dasar dalam keamanan sistem untuk mengelola interaksi antara pengguna dan sumber daya.

A Authentication (Autentikasi):
Proses verifikasi klaim identitas (Apakah Anda benar-benar "Fuadi"?).

A Authorisation (Otorisasi):
Menentukan hak akses setelah identitas terbukti (Apa saja yang boleh Anda akses?).

A Accountability (Akuntabilitas):
Memastikan setiap tindakan dalam sistem dapat dilacak kembali ke entitas yang melakukannya.



AUTENTIKASI: MEMVERIFIKASI IDENTITAS

Entitas vs Identitas: Sistem tidak mengenal "manusia", sistem hanya mengenal "identitas" (seperti username atau ID).

Proses Autentikasi: Verifikasi dilakukan berdasarkan satu atau lebih faktor:

Something you know:

Kata sandi (password), PIN.

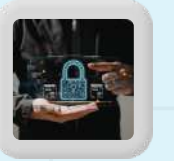
Something you have:

Kartu akses, kode OTP di HP.

Something you are:

Biometrik (sidik jari, wajah)

Multi-Factor Authentication (MFA): Menggabungkan dua atau lebih faktor untuk keamanan ekstra.



OTORISASI & KONTROL AKSES (ACCESS CONTROL)

Terjadi setelah Autentikasi selesai.

Subjek & Objek:

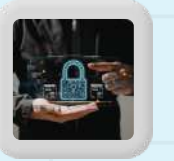
Subjek: Entitas yang meminta akses (User, Proses).

Objek: Sumber daya yang diakses (File, Database, Hardware).

Prinsip Utama:

Least Privilege: Memberikan hak akses sekecil mungkin yang hanya diperlukan untuk tugas tertentu.

Separation of Duties: Membagi tugas sensitif kepada lebih dari satu orang untuk mencegah kecurangan.



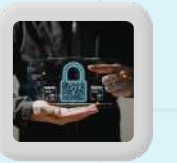
MODEL KONTROL AKSES (DAC & MAC)

DAC (Discretionary Access Control):

1. Akses ditentukan oleh pemilik objek.
2. Sangat fleksibel (Contoh: Manajemen file di Windows/Linux).

MAC (Mandatory Access Control):

1. Akses ditentukan oleh kebijakan sistem (pusat).
2. Berdasarkan label keamanan (Contoh: Rahasia, Sangat Rahasia). Pengguna tidak bisa mengubah izin secara mandiri.



R OLE-BASED ACCESS CONTROL (RBAC)

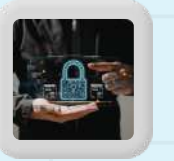
- Model yang paling banyak digunakan di organisasi modern.
- Izin akses diberikan kepada Peran (Role), bukan langsung ke individu.

Keuntungan:

Memudahkan administrasi (jika karyawan pindah divisi, cukup ganti "Role"-nya).

Mengurangi kesalahan pemberian izin akses manual.

Contoh: Role "Dosen" bisa input nilai, Role "Mahasiswa" hanya bisa lihat nilai.



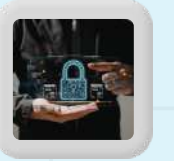
KEBIJAKAN KEAMANAN FORMAL (BELL-LAPADULA & BIBA)

Model Bell-LaPadula (Kerahasiaan):

- No Read Up: Tidak bisa baca data level di atasnya.
- No Write Down: Tidak bisa menulis data ke level di bawahnya (mencegah kebocoran).

Model Biba (Integritas):

- No Read Down: Jangan baca data dari sumber tidak terpercaya.
- No Write Up: Jangan memodifikasi data di level yang lebih tinggi.



A KUNTABILITAS: AUDIT DAN NON-REPUDIATION

Audit Trails (Log): Rekaman kronologis dari semua aktivitas sistem.

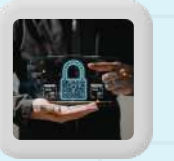
Tujuan Akuntabilitas:

Mendeteksi adanya penyusupan atau perilaku aneh.

Membantu pemulihan sistem setelah terjadi insiden.

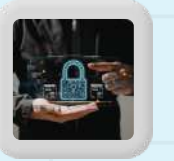
Non-Repudiation (Nir-penyangkalan):

Menjamin bahwa seseorang tidak dapat menyangkal telah melakukan transaksi atau tindakan tertentu (biasanya menggunakan tanda tangan digital).



KESIMPULAN BAGIAN AUTHENTICATION, AUTHORISATION, & ACCOUNTABILITY)

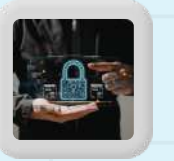
- AAA bukan sekadar fitur, tapi satu kesatuan siklus keamanan.
- Tanpa Autentikasi, kita tidak tahu siapa yang masuk.
- Tanpa Otorisasi, semua orang bisa merusak data.
- Tanpa Akuntabilitas, kita tidak bisa belajar dari kesalahan atau menangkap pelaku kejahatan digital.



FMTS

FORMAL METHODS FOR SECURITY

Referensi: Formal Methods for Security Knowledge Area Version 1.0.0 (David Basin)



INTRODUCTION TO FORMAL METHODS

Mengapa Pengujian Biasa Tidak Cukup?

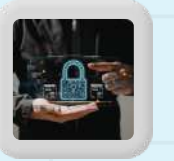
- Pengujian (testing) hanya mengecek sebagian kecil perilaku sistem
- Sistem yang lulus testing belum tentu aman
- Testing hanya bisa menunjukkan ada bug
- Tidak bisa membuktikan bahwa bug keamanan tidak ada

Banyak celah keamanan muncul dari:

- kondisi ekstrem yang jarang diuji
- interaksi kompleks antar komponen
- perilaku penyerang yang tidak terduga

Formal methods hadir untuk:

- menggunakan logika dan matematika
- memberikan pembuktian keamanan
- Tujuan: membuktikan bahwa tidak ada eksekusi sistem yang melanggar properti keamanan



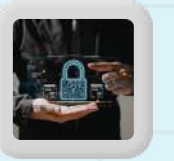
FORMAL SPECIFICATION & MODELLING

Mendefinisikan Sistem dan Keamanan Secara Formal

- Sistem dimodelkan sebagai objek matematis
- Aturan dan perilaku sistem ditulis secara presisi
- Model penyerang didefinisikan secara eksplisit seperti apa yang bisa dilakukan penyerang dan apa yang tidak bisa dilakukan
- Keamanan hanya valid jika model ancaman jelas

Properti keamanan dinyatakan dengan logika formal:

- **Safety properties**
 - sesuatu yang berbahaya tidak pernah terjadi
 - contoh: penyerang tidak pernah mendapatkan kunci rahasia
- **Hyperproperties**
 - keamanan dilihat dari hubungan antar banyak eksekusi
 - contoh: kerahasiaan dan noninterference
 - Hyperproperty penting karena kebocoran data sering tidak terlihat dari satu eksekusi



VERIFICATION TECHNIQUES

M Model Checking

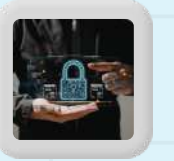
- Mengecek semua kemungkinan eksekusi model sistem
- Dilakukan secara otomatis oleh tool
- Jika properti dilanggar maka sistem menghasilkan counterexample
- Cocok untuk sistem dengan state terbatas

T Theorem Proving

- Membuktikan properti keamanan secara matematis
- Menggunakan logika yang sangat ekspresif
- Memberikan jaminan keamanan paling kuat
- Membutuhkan keahlian dan interaksi manusia

S Static Analysis

- Menganalisis kode tanpa menjalankannya
- Skalabel untuk kode yang sangat besar
- Fokus pada kerentanan umum (misalnya bug memori)
- Bisa menghasilkan false positives



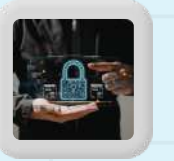
CASE STUDY: SECURITY PROTOCOLS

Verifikasi Protokol Komunikasi

- Protokol keamanan berjalan di jaringan dengan penyerang aktif
- Kesalahan kecil dalam desain → celah besar
- Testing sulit menangkap:
 - urutan pesan yang tidak normal
 - manipulasi pesan oleh penyerang
- Formal methods digunakan untuk membuktikan:
 - autentikasi benar
 - pesan tetap rahasia
 - tidak ada penyamaran identitas
- Tools formal mampu:
 - menemukan serangan tersembunyi
 - membuktikan tidak adanya serangan tertentu

Contoh protokol:

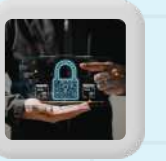
- **Transport Layer Security (TLS)**
 - diverifikasi untuk mencegah man-in-the-middle
 - fokus pada keamanan proses handshake
- **Security Assertion Markup Language (SAML)**
 - diverifikasi agar klaim identitas tidak bisa dipalsukan



CASE STUDY: OPERATING SYSTEMS

Pembuktian Keamanan Microkernel seL4

- Sistem operasi adalah fondasi seluruh sistem
- Bug di kernel → kontrol penuh atas sistem
- Testing kernel sangat sulit dan tidak mencakup semua kemungkinan eksekusi
- Microkernel seL4
 - microkernel yang diverifikasi secara matematis
 - menggunakan interactive theorem proving
- Dibuktikan secara formal melalui isolasi memori antar proses dan kontrol akses yang benar
- Buktinya mencakup spesifikasi tingkat tinggi dan implementasi kode C
- Menunjukkan bahwa formal methods praktis dan cocok untuk sistem keamanan-kritis



Thank You